Course 1 (Prompt Engineering)

Lecture 1 -

1. Introduction to Prompt Engineering

Definition: Prompt Engineering is the process of writing clear instructions to get useful responses from AI.

Example:

- Poor prompt: "Explain AI."
- Better prompt: "Explain Machine Learning in simple English with three real-world examples."

Key point: Better instructions can help produce better responses.

2. Definition of a Prompt

Definition: A prompt is the instruction, question, or input given to an AI model.

Example:

"Explain Machine Learning in simple English, with a definition, three examples, and exam points."

Key point: A prompt tells AI what we want it to do.

3. Core Elements of a Good Prompt

Three elements discussed in our lesson are:

- **Definition:** What is the task?
- **Purpose:** Why do we need it?
- **Goal:** What result do we expect?

Example:

- Definition: Explain inheritance in Java.
- Purpose: I am preparing for my university exam.
- Goal: Provide a short program, output, and three revision points.

4. Persona

Definition: Persona means assigning a role to AI to guide its response style.

Example:

"Act as an experienced Java teacher."

Use: It helps tailor the response to a particular role or audience.

5. Context

Definition: Context is the background information that helps AI understand our situation.

Example:

“I am preparing for my university examination.”
Use: It helps AI make the response more relevant to our needs.

6. Constraints

Definition: Constraints are rules or limits given to AI.
Examples:

- Use simple English.
- Write exactly 100 words.
- Include two real-world examples.
- Avoid complex mathematical equations.

Key point: Constraints help control the response, though they are not always followed perfectly.

7. Tone

Definition: Tone is the style or manner in which AI communicates.
Common tones: Formal, informal, friendly, professional, academic, and humorous.
Example:
“Explain Artificial Intelligence in a friendly and conversational tone.”
Key point: Tone changes the communication style, not necessarily the underlying facts.

8. Structure

Definition: Structure specifies how the response should be organized.
Common formats: Headings, bullet points, numbered steps, tables, paragraphs, and code blocks.
Example:
“Explain AI using headings, bullet points, and a summary table.”
Key point: Structure makes answers easier to read and understand.

9. Examples in Prompting

Examples help AI understand the expected pattern of a task.

- **Zero-shot prompting:** No examples are provided.
- **One-shot prompting:** One example is provided.
- **Few-shot prompting:** Multiple examples are provided.

Example — Translation:

- Zero-shot: “Translate ‘Good morning’ into Hindi.”
- One-shot: “Good morning → सुप्रभात। Now translate ‘Thank you’ into Hindi.”
- Few-shot: Provide several translation examples before asking for a new translation.

Key point: Zero = no examples, One = one example, Few = multiple examples.

10. Large Language Models (LLMs)

Definition: Large Language Models are AI models trained on large amounts of data to learn language patterns and generate text.
Examples: GPT-based models and other generative language models.
Example:
Input: "The Sun rises in the ..."
The model may predict "east" as the next token.
Key point: LLMs generate text by repeatedly predicting and selecting subsequent tokens.

11. Tokenization

Definition: Tokenization is the process of dividing text into smaller units called tokens.
A token can be a complete word, part of a word, or punctuation.
Example:
Text: I love Machine Learning.
A tokenizer could split this into units corresponding to:
I | love | Machine | Learning | .
This is only an illustrative split. The exact tokens depend on the tokenizer.
Key point: One word does not always equal one token.

12. Output Token Limits

Definition: The output token limit is the maximum number of tokens a model can generate in a response.
Example:
If a detailed explanation requires more tokens than the allowed limit, the response may end before the explanation is complete.
Key point: A higher output limit allows a longer response, but does not guarantee a better one.

13. Context Window

Definition: A context window is the amount of input and conversation context a model can process at one time.
Example:
When working with a large document, the model must process the relevant text within its context capacity.
Key point: Context window and output token limit are different:

- Context window: Input and conversation context the model can handle.
- Output token limit: Maximum tokens generated in the response.

14. Temperature

Definition: Temperature controls how sharply or broadly token

probabilities influence selection during text generation.
Example:
Prompt: “The weather today is ...”

- Low temperature: Usually more predictable choices.
- Higher temperature: Usually more variation in choices.

Key point: Temperature affects sampling variability; it does not guarantee creativity or correctness.

15. Top-K Sampling

Definition: Top-K sampling restricts the next-token candidates to the K tokens with the highest probabilities.
Example:
For “The weather today is ...”:

- Sunny: 40%
- Cloudy: 30%
- Rainy: 15%
- Other candidates: Lower probabilities

If K = 2, only “sunny” and “cloudy” remain eligible in this example. The model selects between them based on their adjusted probabilities.
Key point: Top-K controls the number of candidate tokens.

16. Top-P Sampling (Nucleus Sampling)

Definition: Top-P sampling keeps the smallest set of highest-probability candidates whose cumulative probability reaches the specified threshold P.
Example:

- Sunny: 40%
- Cloudy: 30%
- Rainy: 15%
- Mango: 7%
- Grape: 3%

If P = 0.80:

- First two candidates total 40% + 30% = 70%.
- Add the third candidate: 70% + 15% = 85%.
- The threshold has been reached, so candidate selection stops.

Key point: Top-P controls the cumulative probability mass of the candidate set.

17. Top-K vs Top-P

Top-K	Top-P
Uses a fixed number of candidates.	Uses a cumulative probability threshold.
Example: K = 3 keeps three candidates.	Example: P = 0.80 keeps enough candidates to reach 80% or more.
Candidate count is fixed.	Candidate count can vary.

Easy way to remember:

- Top-K = How many candidates?
- Top-P = How much cumulative probability?

18. Combining Decoding Parameters

Definition: Combining decoding parameters means using settings Such as temperature, Top-K, and Top-P together to influence text generation.

Example:

Prompt: "Write a short story about a robot."

- Lower temperature: Generally more predictable text.
- Higher temperature: Generally more variation.
- Top-K: Restricts the number of eligible candidates.
- Top-P: Restricts candidates according to cumulative probability.

Key point: Temperature affects the probability distribution; Top-K and Top-P restrict the candidate set.

The effect depends on the model and configuration.

19. Prompt Engineering Best Practices

To write an effective prompt:

1. Be clear and specific.
2. Provide relevant context.
3. Assign a persona when useful.
4. Set constraints.
5. Specify the output structure.
6. Provide examples when needed.
7. State the expected goal.
8. Verify important facts in the response.

Final Example:

"Act as an experienced computer science professor. I am an M.Tech CSE student preparing for my university examination. Explain Object-Oriented Programming in simple English, include a definition, four main principles, real-world examples, and a short Java program with output. Use clear headings and a comparison table wherever appropriate. If any important fact is uncertain, clearly mention the uncertainty."

Key point: A good prompt combines clear instructions, relevant context, suitable constraints, and an expected output format.

Quick Revision

- **Prompt Engineering:** Designing effective instructions for AI.
- **Prompt:** Input or instruction given to AI.
- **Persona:** Role assigned to AI.
- **Context:** Background information.
- **Constraints:** Rules and limits.
- **Tone:** Communication style.
- **Structure:** Output organization.
- **LLM:** Large Language Model.
- **Tokenization:** Dividing text into tokens.
- **Output Token Limit:** Maximum generated tokens.
- **Context Window:** Input and conversation context capacity.
- **Temperature:** Influences sampling variability.
- **Top-K:** Fixed number of candidate tokens.
- **Top-P:** Probability-based candidate selection.
- **Best Practices:** Clear instructions, useful context, suitable formatting, and fact-checking.

Lecture 2 -

1. Zero-Shot Prompting

Definition: Zero-shot prompting means asking an AI model to perform a task without providing any examples.

Example: Movie Review Classification

Prompt:

```
Task: Classify the movie review as Positive or Negative.

Review: The movie was excellent and entertaining.

Output: Positive
```

Here, the task is described directly without showing any example first.

Advantages:

- Fast and simple.
- Requires fewer tokens.
- Useful for common tasks.

Limitations:

- Responses may be inconsistent.
- Complex tasks may require examples.

2. Advantages of Zero-Shot Prompting

1. **Efficiency:** No examples are needed.

2. **Flexibility:** Useful for many different tasks.
3. **Accessibility:** Easy for beginners to use.
4. **Simplicity:** Requires a shorter prompt.

Key Point: Zero-shot prompting is a good starting point for simple tasks.

3. Designing a Good Zero-Shot Prompt

A good prompt should contain three important elements:

- **Clear Task Definition:** Explain exactly what the AI must do.
- **Constraints:** Specify rules, limits, or allowed outputs.
- **Structured Instructions:** Present instructions in an organized way.

Example:

```
Classify the movie review as Positive or Negative.

Review: The movie was excellent.

Return only one label:
Positive or Negative.
```

The output constraint helps prevent unnecessary explanations.

4. Limitations of Zero-Shot Prompting

Zero-shot prompting may not work consistently when:

- The task is complex.
- The expected output format is specific.
- The task requires learning a particular pattern from examples.
- The instruction is ambiguous.

In such cases, One-Shot or Few-Shot Prompting may be more useful.

5. One-Shot Prompting

Definition: One-shot prompting means providing one example before asking the AI to perform a similar task.

Example:

```
Review: The movie was fantastic.
Label: Positive

Now classify this review:
Review: The movie was boring.
Label:
```

The first example demonstrates the expected pattern. The model uses that pattern to classify the new review.

Key Point: One-Shot = One Example.

6. Few-Shot Prompting

Definition: Few-shot prompting means providing multiple examples in a prompt so that the model can follow the demonstrated pattern.

Example: Pizza Order Conversion to JSON

```
Example 1:
Order: One small cheese pizza.
Output:
{
  "size": "small",
  "topping": "cheese"
}

Example 2:
Order: One large mushroom pizza.
Output:
{
  "size": "large",
  "topping": "mushroom"
}

Now convert this order:
Order: One medium paneer pizza.
```
Expected output:
```
{
  "size": "medium",
  "topping": "paneer"
}
```
The examples demonstrate how natural-language pizza orders should be converted into structured JSON.
Key Point: Few-Shot = Multiple Examples.

7. Benefits of Few-Shot Prompting

1. **Explicit Expectations:** Examples show the model what output is required.
2. **Pattern Recognition:** The model can follow the demonstrated format.
3. **Consistency:** Similar inputs can receive similarly structured outputs.
4. **Better Task Guidance:** Useful for complex or specialized tasks.

Few-shot prompting guides the model through examples; it does not retrain the underlying model.

8. Applications of Few-Shot Prompting

Few-shot prompting is useful in:

- **Data Extraction:** Extracting names, dates, or other fields.
- **Data Formatting:** Converting information into JSON.
- **Workflow Automation:** Producing consistently structured outputs.
- **Agent Pipelines:** Helping AI systems follow expected input-output formats.

Trade-off: More examples can improve guidance, but they also make the prompt longer and consume more tokens.

9. Why Use JSON in Prompting?

Definition: JSON (JavaScript Object Notation) is a structured data format that represents information using key-value pairs, arrays, and nested objects.

Example:

```
{
  "pizza_size": "large",
  "topping": "mushroom"
}
```

Advantages:

- Organized and structured data.
- Easy for programs to process.
- Useful for APIs and software integration.
- Supports automated workflows.

Key Point: JSON helps AI return information in a consistent, machine-readable format.

Limitations of Few-Shot Prompting

1. Long Prompts: More examples consume more tokens.
2. Incorrect Examples: Wrong examples can lead to incorrect outputs.
3. Limited Variety: Too few variations may reduce performance on unfamiliar inputs.
4. Edge Cases: Incomplete or unusual inputs can be difficult to handle.
5. No Guarantee of Accuracy: AI can still produce incorrect answers.
6. Example Selection: Choosing relevant, diverse, high-quality examples takes time.

10. Choosing Good Examples for Few-Shot Prompting

The quality of examples strongly influences the usefulness of the output.

A. Relevance

Examples should closely match the task.

Example: Use pizza-order examples when the task is to convert pizza orders into JSON.

B. Diversity

Include different valid input variations so the model can handle different requests.

Example: Include small, medium, and large pizza orders with different toppings.

C. Quality

Examples must be accurate, clear, and consistently formatted.

Example: Every pizza order should follow the same JSON structure. Incorrect examples may encourage incorrect or inconsistent outputs.

D. Edge Cases

Include unusual or difficult inputs that may occur in real-world use.
Example: An order with no topping specified or an unfamiliar pizza size. The prompt should clarify how such cases must be handled rather than encouraging the model to guess.

11. Few-Shot Prompting as Test-Case Design

Designing few-shot prompts is similar to preparing test cases for software.

- Examples describe expected behavior.
- Different inputs test different situations.
- Consistent outputs demonstrate the required format.
- Edge cases check how unusual inputs should be handled.

Key Point: Carefully selected examples help communicate the intended behavior to an AI model.

Quick Revision Sheet

Concept	Meaning
Zero-Shot Prompting	Performing a task without examples.
One-Shot Prompting	Providing one example.
Few-Shot Prompting	Providing multiple examples.
Clear Task Definition	Clearly stating what the AI must do.
Constraints	Defining rules and output limits.
Structured Instructions	Organizing the prompt clearly.
JSON	A structured, machine-readable data format.
Relevance	Choosing examples related to the task.
Diversity	Covering different input variations.
Quality	Using correct and consistently formatted examples.
Edge Cases	Including unusual or difficult inputs.

Important Questions for Students

1. What is zero-shot prompting?
2. What are the advantages and limitations of zero-shot prompting?
3. Differentiate between zero-shot, one-shot, and few-shot prompting.
4. What is the purpose of JSON in AI applications?

5. Explain relevance, diversity, quality, and edge cases in few-shot prompting.
6. How can few-shot prompting help with data extraction and workflow automation?
7. Why should examples be accurate and consistent?

Lecture 3 -

1. Introduction to Prompting Approaches

The three main prompting approaches covered in this lecture are:

1. **System Prompting**
2. **Contextual Prompting**
3. **Role Prompting**

Each technique has a different primary purpose, and they can be combined to produce more relevant and consistent AI responses.

2. System Prompting

Definition: System prompting provides the overall instructions, purpose, rules, and behaviour that guide an AI model throughout a conversation.

Key features:

- Defines the AI's overall behaviour and response style.
- Sets boundaries for what the AI should address or avoid.
- Can specify output formats such as JSON or Markdown.
- Provides foundational instructions before user interactions begin.

Example: Movie Review Classification

Instruction:

```
Classify movie reviews as Positive, Neutral, or Negative.
Return a JSON object containing the movie name and
sentiment.
```

Input:

```
Movie: Inception
Review: A brilliant film with excellent performances.
```

Expected output:

```
{
  "movie_name": "Inception",
  "sentiment": "Positive"
}
```

3. Types of System Prompts

A. Basic System Prompts

Definition: Basic system prompts provide clear and concise instructions for a particular task.

Example:

```
Classify movie reviews as Positive, Neutral, or Negative.
Return the result in JSON format.
```

Purpose: To make the task and expected output clear.

B. Guardrail Prompts

Definition: Guardrail prompts establish boundaries that help keep the AI focused on its intended purpose.

Example:

```
You are a movie-review analysis assistant.
Answer questions related to movie reviews.
Politely redirect unrelated requests.
```

Purpose: To prevent off-topic responses and maintain task focus.

C. Structured Output Instructions

System instructions can also specify a consistent response format, such as JSON or Markdown.

Example:

```
{
  "movie_name": "Inception",
  "sentiment": "Positive"
}
```

Purpose: To make responses easier for applications to parse and process.

4. Contextual Prompting

Definition: Contextual prompting provides relevant background information and specific details that help an AI understand the situation and produce more appropriate responses.

Key benefits:

- Makes responses more relevant to the task.
- Helps the AI understand the intended audience and purpose.
- Provides background information needed for the task.

Example: Retro Games Blog

Prompt:

```
I am writing for a retro games blog.
Suggest three article topics about retro gaming.
Include a brief description for each topic.
```

Possible output:

1. **Evolution of Arcade Cabinet Design:** Explore the development from early wood and metal cabinets to the neon-lit designs of the 1980s.
2. **Iconic Arcade Games of the 80s:** Discuss the history, influence, and lasting appeal of famous arcade games.
3. **The Retro Revival of Pixel Art:** Explore the renewed popularity of pixel art in modern gaming.

Purpose: The background information helps tailor the suggestions to a retro gaming audience.

5. Role Prompting

Definition: Role prompting assigns a specific role, identity, or professional perspective to an AI model to guide its response style and approach.

Example:

```
Act as an experienced retro gaming blogger.
Suggest three engaging article topics for a retro games blog.
Use a nostalgic and informative tone.
```

Purpose: To guide the AI's expertise, voice, and communication style.

Common roles:

- Teacher
- Software Developer
- Researcher
- Data Analyst
- Technical Writer

6. Difference Between System, Contextual, and Role Prompting

Technique	Main purpose	Example
System Prompting	Defines overall behaviour and rules	Classify reviews and return JSON.
Contextual Prompting	Provides background and task-specific details	The content is for a retro games blog.
Role Prompting	Assigns an identity or perspective	Act as an experienced gaming blogger.

7. Using All Three Techniques Together

These approaches can be combined because each serves a different purpose.

Combined Example:

```
System:
Follow the requested output format and remain focused on the task.

Context:
The content is for a retro games blog aimed at gaming enthusiasts.

Role:
Act as an experienced retro gaming blogger.

Task:
```

```
Suggest three article topics about 1980s arcade games.
Provide a brief description of each topic.
```
Here:

- **System** defines the overall rules.
- **Context** explains the background and audience.
- **Role** defines the writing perspective.
- **Task** specifies the requested output.

Quick Revision

- **System Prompting:** Sets foundational instructions and behavioural boundaries.
- **Basic System Prompt:** Gives clear task instructions.
- **Guardrail Prompt:** Establishes boundaries and keeps the AI on topic.
- **Structured Output:** Specifies formats such as JSON or Markdown.
- **Contextual Prompting:** Supplies relevant background information.
- **Role Prompting:** Assigns a specific identity or perspective.
- **Combined Prompting:** Uses these techniques together for more targeted responses.

Final Takeaway: System prompting defines the overall rules, contextual prompting supplies the background, and role prompting guides the AI's identity and response style.

Lecture 4 -

1. Evolution of Language Models

Language models have evolved through major improvements in model architecture, training methods, and prompting techniques.

Key points:

- Transformers introduced a major architectural breakthrough.
- GPT-2 and GPT-3 demonstrated the potential of large-scale pre-training.

- GPT-2 improved text generation and demonstrated that large Transformer-based language models could perform multiple language tasks without task-specific training
- GPT-3 helped popularize few-shot prompting, where examples guide the model's output.

2. Chain-of-Thought (CoT) Prompting

Definition: Chain-of-Thought prompting encourages an AI model to work through a problem in a sequence of intermediate steps.

Purpose: It can help with complex tasks that require multiple reasoning steps.

Types of CoT Prompting

- **Standard CoT:** Encourages step-by-step problem solving.
- **Few-Shot CoT:** Provides examples that demonstrate the expected reasoning approach.
- **Zero-Shot CoT:** Encourages step-by-step problem solving without providing reasoning examples.(Hint Is Give also)

3. Principles of Chain-of-Thought Prompting

- Break complex problems into smaller steps.
- Organize the solution in a logical sequence.
- Make the solution process easier to follow.
- Use intermediate steps to support the final answer.

4. Benefits of Chain-of-Thought Prompting

- **Accuracy:** Can improve performance on some multi-step tasks.
- **Interpretability:** Makes the solution easier to inspect.
- **Robustness:** Can help with certain complex problems.
- **Ease of Implementation:** Can be applied through prompting.
- **Problem Solving:** Supports tasks requiring multiple steps.

These benefits depend on the task and model; CoT does not guarantee correct answers.

5. Chain-of-Thought Example: Age Difference Problem

The lecture uses an age-difference problem to demonstrate step-by-step reasoning.

Approach:

1. Identify the information given in the problem.
2. Determine the relationship between the ages.
3. Solve the problem step by step.
4. Check the result and provide the final answer.

Age Difference Problem

```
Question: “When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner? Let’s think step by step.”

Step 1: Find the partner’s age when I was 3.
Partner’s age = 3 × 3 = 9 years

Step 2: Calculate how much my age has increased.
Age increase = 20 − 3 = 17 years
```

```
Step 3: Calculate the partner's current age.
Partner's current age = 9 + 17 = 26 years

Final Answer: 26 years old.
```
Note: The exact ages and numerical details of the video's example were not available in the extracted lecture summary, so they have not been invented here.

6. Reasoning Examples in Prompting

Providing worked examples can demonstrate how the model should approach a new problem.

Key points:

- Examples show the expected problem-solving pattern.
- Consistent examples can improve output consistency.
- The quality of the examples influences their usefulness.

7. Automatic Chain-of-Thought (Auto-CoT)

Definition: Auto-CoT automates the creation of example reasoning sequences for use in prompting.

Basic workflow:

1. Group similar questions using clustering.
2. Select representative questions.
3. Generate reasoning examples for those questions.
4. Use the selected examples to guide the model on other questions.

Purpose: Reduce the manual effort required to create CoT examples.

8. Tree of Thoughts (ToT)

Definition: Tree of Thoughts structures problem solving as multiple possible reasoning paths that branch from one another.

Key points:

- Explores multiple possible approaches.
- Allows the system to evaluate different paths.
- Supports backtracking when a path is not useful.
- Can be useful for complex planning and problem-solving tasks.

Example: Tree of Thoughts (ToT)

```
Question: You have only 3 days to prepare for an exam. How
should you plan your studies?
Step 1: Explore Different Paths 🌳

Main Problem
Prepare for the exam in 3 days.

Path A
```

```
Study difficult topics first.

Path B
Study important topics first.

Path C
Read every topic equally.
```

Step 2: Evaluate the Paths –
Path A: Useful if difficult topics need more time.
Path B: Useful for prioritizing high-weightage topics.
Path C: Covers the syllabus broadly but may not prioritize important topics.

Step 3: Choose a Suitable Path
For an exam in only 3 days, Path B may be a practical starting point. Study important topics first, then cover difficult areas and revise.
Key Point: Tree of Thoughts explores multiple possible solution paths, evaluates them, and selects a suitable one instead of following only one path from the beginning.

9. Chain of Thought vs. Tree of Thoughts

Chain of Thought	Tree of Thoughts
Follows a largely linear sequence of steps.	Explores branching reasoning paths.
Focuses on one developing path.	Can compare several possible paths.
Suitable for many step-by-step tasks.	Useful when multiple alternatives need exploration.

10. Self-Consistency

Definition: Self-Consistency generates multiple possible reasoning paths and uses their agreement or another selection method to identify a more reliable answer.
Basic workflow:

1. Generate multiple solution paths.
2. Compare their final answers.
3. Select the answer supported by the chosen consistency method.

Benefit: Reduces dependence on a single reasoning path and can improve accuracy on suitable tasks.

11. Greedy Decoding and Self-Consistency

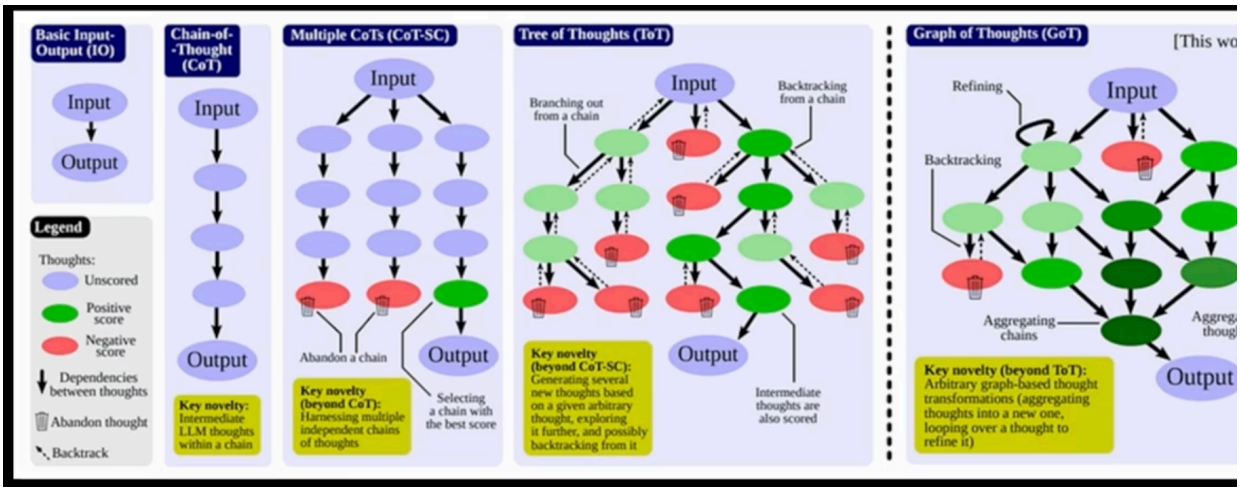
Greedy Decoding: At each generation step, selects the token with the

highest probability under the model's current decoding distribution.
Limitation: A single locally preferred choice may lead to a less useful overall solution.
Self-Consistency: Samples multiple possible reasoning paths and compares their results instead of relying on just one path.

12. Graph of Thoughts (GoT)

Definition: Graph of Thoughts represents reasoning as a graph of interconnected ideas or thought units.
Key points:

- Different ideas can connect to one another.
- Multiple solution paths can be explored.
- Results from different paths can be combined.
- It can support complex planning, scientific reasoning, and multi-agent collaboration.



CoT vs ToT vs GoT

Feature	CoT	ToT	GoT
Full Form	Chain of Thought	Tree of Thoughts	Graph of Thoughts
Structure	Linear steps	Branching paths	Interconnected ideas
Approach	Follows steps sequentially	Explores multiple paths	Connects and combines ideas
Best For	Step-by-step problems	Comparing alternatives	Complex problems with connected ideas

13. Chain of Thought vs. Graph of Thoughts

Chain of Thought	Graph of Thoughts

Primarily linear reasoning.	Interconnected reasoning structure.
Develops a sequence of steps.	Connects and combines different ideas.
Easier to follow for straightforward tasks.	Supports more complex relationships between ideas.

14. Skeleton of Thought (SoT)

Definition: Skeleton of Thought is a prompting technique that first creates an outline and then expands its sections, potentially in parallel.
Purpose: Improve response-generation efficiency while maintaining an organized, coherent answer.

Workflow

1. **Create an Outline:** Generate the main points or sections.
2. **Parallel Expansion:** Expand different sections, potentially at the same time.
3. **Final Aggregation:** Combine the expanded sections into one coherent response.

Key benefit: Separates the planning of an answer from the expansion of its content.

Quick Revision Sheet

Technique	Main Idea
Transformer	A major neural network architecture for language models.
Chain of Thought (CoT)	Step-by-step problem solving.
Zero-Shot CoT	Step-by-step prompting without reasoning examples.
Few-Shot CoT	Reasoning examples guide the model.
Auto-CoT	Automates the generation of reasoning examples.
Tree of Thoughts (ToT)	Explores branching reasoning paths.
Self-Consistency	Compares multiple sampled reasoning paths.
Greedy Decoding	Selects the highest-probability next token at each step.
Graph of Thoughts (GoT)	Represents reasoning as interconnected ideas.

Skeleton of Thought (SoT)	Creates an outline, expands sections, then combines them.

Final Takeaway

Prompting techniques have progressed from giving direct instructions to structuring more complex problem-solving processes. **CoT** emphasizes sequential steps, **ToT** explores branches, **GoT** connects ideas, **Self-Consistency** compares multiple candidate solutions, and **SoT** organizes an answer through outlining and expansion.

Lecture - 5

ReAct Prompting

1. Introduction to ReAct Prompting

Definition: ReAct stands for **Reasoning and Acting**.
It is a framework that enables AI agents to combine reasoning with actions, such as using tools or gathering external information, to solve problems.

Main purpose: To help AI handle complex, multi-step tasks by interacting with its environment and adapting to new information.
Key components:

- Reasoning
- Acting
- Information gathering
- Plan updating

2. Core Components of ReAct

A. Reasoning

The model determines what information or action is needed to solve the problem.

B. Information Gathering

The model receives information from the action it performed.

C. Action

The model performs an action, such as searching for information or using a tool.

D. Observation

Check and understand the results after an action."

E. Plan Updating

The model uses the new information to decide what to do next.
Key point: These components work together in an iterative process.

3. The ReAct Cycle

Definition: The ReAct cycle is an iterative process in which an AI agent reasons, takes an action, observes the result, and updates its next step.

Three main stages

1. **Think:** Determine the next step needed to solve the task.
2. **Act:** Perform an action, such as retrieving external information.
3. **Observe:** Examine the result and use it to determine the next step.

The cycle can repeat until the task is completed.

Cycle:

Think → Act → Observe → Update Plan → Repeat if needed

4. ReAct Prompting

Definition: ReAct prompting instructs an AI model to combine reasoning with actions, including tool use, and refine its response using the results obtained.

How it works:

1. Receive the user's question.
2. Identify what information is needed.
3. Take an appropriate action.
4. Observe the result.
5. Update the plan if necessary.
6. Provide the final answer when sufficient information is available.

5. ReAct Example from the Lecture

The lecture demonstrates a task that combines **external information retrieval and symbolic computation**.

```
The process is:
Reason: Identify the information required to solve the task.
Retrieve: Use an external source to obtain relevant
information.
Observe: Examine the retrieved information.
Compute: Apply symbolic computation where required.
Answer: Use the gathered information and computation to
produce the result.
```

Important note: The extracted lecture summary identifies this worked-example approach but does not preserve the exact question or numerical details. Therefore, they have not been invented here.

6. Advantages of ReAct Prompting

- **Active Problem Solving:** Combines reasoning with actions.
- **Information Retrieval:** Can obtain information from external sources.
- **Adaptability:** Can update its next step using new observations.
- **Multi-Step Tasks:** Supports problems requiring several actions.

- **Tool Integration:** Can work with external tools and computational systems.

7. ReAct vs. Act-Only Prompting

Feature	ReAct Prompting	Act-Only Prompting
Main approach	Combines reasoning and acting	Focuses on actions
Process	Think → Act → Observe	Action-focused process
Information use	Can use observations to update the plan	Depends on the action strategy
Complex tasks	Combines reasoning and tool use	May lack the same explicit reasoning loop
Lecture evaluation	ReAct generally performed better on the evaluated challenging tasks	Used as a comparison approach

8. Evaluation of ReAct

The lecture discusses evaluations on knowledge-intensive tasks, including:

- **HotpotQA:** A dataset used to evaluate question answering that can require information from multiple sources.
- **FEVER:** A dataset used for fact verification, where claims are checked against evidence.

The lecture reports that ReAct and ReAct Plus outperformed other approaches on the discussed challenging tasks by combining reasoning and acting.

Key point: Performance depends on the task, model, tools, and evaluation setup.

HotpotQA

Definition: HotpotQA is a dataset used to test whether an AI can answer questions that require combining information from multiple documents.

```
Example:
Question: Who is the spouse of the author of Harry Potter?

Step 1: Identify the author of Harry Potter → J. K. Rowling.
Step 2: Find her spouse → Neil Murray.
```

```
Final Answer: Neil Murray.
```
Purpose: To test whether AI can connect multiple pieces of information to answer a question.
This is an illustrative example, not necessarily an exact question from the dataset.

2. FEVER
Full Form: Fact Extraction and VERification.

Definition: FEVER is a dataset used to test whether an AI can verify statements using evidence.
```
Example:

Claim: "The Eiffel Tower is located in Berlin."
The AI checks the claim against reliable evidence.
```

Label	Meaning	Example
SUPPORTED	Evidence supports the claim.	The Eiffel Tower is in Paris.
REFUTED	Evidence contradicts the claim.	The Eiffel Tower is in Berlin.
NOT ENOUGH INFO	Evidence is insufficient to decide.	The available evidence does not establish the tower's location.

Correct result for the claim: REFUTED, because the Eiffel Tower is in Paris, France—not Berlin.

Quick Revision

Term	Meaning
ReAct	Reasoning + Acting
Reasoning	Determining the next step
Acting	Performing an action or using a tool
Observation	Examining the result of an action
Plan Updating	Adjusting the next step based on new information
ReAct Cycle	Think → Act → Observe
HotpotQA	Multi-source question-answering benchmark

FEVER	Fact-verification benchmark

Final takeaway: ReAct combines reasoning, tool use, and observation in an iterative loop, helping AI agents solve multi-step tasks using new information.

Lecture - 6

1. Introduction to Automatic Prompt Engineering

Definition: Automatic Prompt Engineering (APE) is an approach that uses language models to generate, evaluate, and improve prompts automatically.
Purpose: To find effective prompts without manually writing and testing every possible variation.

2. How Automatic Prompt Engineering Works

APE follows an iterative process:

1. **Generate:** Create multiple prompt variations.
2. **Evaluate:** Test the prompts on the required task.
3. **Refine:** Improve prompts based on their results.
4. **Repeat:** Continue testing until a suitable prompt is found.

Key Point: APE uses repeated testing and improvement to optimize prompts.

3. Automatic Prompt Engineer App

Definition: The Automatic Prompt Engineer App automates the process of generating, testing, and selecting effective prompts.
Main Steps:

- **Instruction Generation:** Generate different prompt for candidate.
- **Execution Testing:** Run the prompts and evaluate their outputs.
- **Optimal Selection:** Select the best-performing prompt according to the evaluation criteria.
- **Selection C**ontinuous Improvement

4. Benefits of Automatic Prompt Engineering

1. **Rapid Iteration:** Tests multiple prompt variations quickly.
2. **Optimization:** Helps identify effective prompts.
3. **Time Savings:** Reduces repetitive manual prompt testing.
4. **Diverse Applicability:** Can be used across different tasks and applications.
5. **Continuous Improvement:** Prompts can be refined using evaluation results.

5. Metallica T-Shirt Orders — Example

Purpose: This example demonstrates why a chatbot must understand different expressions of the same customer intent.
Customers may express a similar request in different ways.

Illustrative variations based on the lecture's example:
"I want a Metallica T-shirt."
"Can I order a Metallica tee?"
"Please get me a Metallica shirt."
Although the wording differs, the underlying intent is similar.

Which is better Prompt Which ask size !

How prompt engineering helps:

- Exposes the chatbot to diverse expressions.
- Helps it recognize the customer's intended request.
- Improves consistency across different phrasings.
- Reduces the risk of misunderstanding customer orders.

Key Point: A well-designed chatbot should recognize the intended meaning, not depend only on one exact sentence.
Note: These sentences illustrate the lecture's Metallica T-shirt example; the exact original wording was not preserved in the extracted video summary.

6. Emotion Prompting

Definition: Emotion prompting uses emotional cues or emotional context in a prompt to influence how an AI model approaches a task and generates its response.
Purpose: To encourage more appropriate responses and potentially improve performance on certain tasks.

Illustrative Example:
Without emotional context:
"Help me prepare for my exam."
With emotional context:
"I am worried about my exam. Please help me prepare carefully and explain the important concepts clearly."
The second prompt communicates the user's emotional state and requests a suitable response.

Key Point: Emotional cues may influence model responses, but they do not guarantee better reasoning or accuracy. Use emotional context when

it is relevant to the task.

Benefits of Emotion Prompting

1. Personalized Responses: Adapts responses to the user's emotional situation.
2. Supportive Communication: Encourages more empathetic and reassuring language.
3. Better Engagement: Can make interactions feel more relevant to the user.
4. Clearer Guidance: Helps tailor explanations to users experiencing stress or confusion.

Examples of applications:

1. Education: A student can express exam anxiety and ask for explanations with simple examples.
2. Customer Support: A polite and clear response can be prepared for a frustrated customer.
3. Healthcare Information: Information can be explained in a simple and reassuring way.
4. Personal Assistance: It can help tailor guidance to someone's situation when making decisions.

Important Point: Emotion Prompting uses emotional context to guide AI responses, but adding emotional cues does not automatically improve accuracy.

7. Best Practices for Prompt Improvement

- Generate multiple prompt variations.
- Test prompts on relevant examples.
- Evaluate outputs using clear criteria.
- Prefer prompts that perform consistently.
- Refine prompts when results are unsatisfactory.
- Include diverse ways of expressing the same intent.
- Use emotional context appropriately when relevant.

Quick Revision

Topic	Meaning
Automatic Prompt Engineering (APE)	Automatically generates and evaluates prompt variations.
Prompt Generation	Creates candidate prompts.

Prompt Evaluation	Tests and measures prompt performance.
Prompt Refinement	Improves prompts based on results.
Automatic Prompt Engineer App	Automates generation, testing, and selection.
Metallica T-Shirt Example	Demonstrates recognizing the same intent across different expressions.
Emotion Prompting	Uses emotional cues to guide responses.
Iterative Improvement	Repeats testing and refinement to improve results.

Final Takeaway: Automatic Prompt Engineering improves prompts through generation, evaluation, and refinement, while emotion prompting uses relevant emotional cues to guide the response.

Lecture - 7 (Prompt Engineering for Coding)

1. Creating Effective Code Prompts

Definition: A code prompt is an instruction given to an AI model to generate, explain, translate, optimize, or debug code.
A good code prompt should specify:

- **Task Description:** What should the code do?
- **Programming Language:** Which language should be used?
- **Error Handling:** How should errors and invalid inputs be handled?
- **Style and Documentation:** Should the code contain comments and follow a particular coding style?

Example:

```
Write a Python script to count files in a directory.
Include error handling and comments.
Explain how to run the script.
```

Key Point: Clear instructions help the AI generate code that better matches the requirements.

2. Bash Script Example — File Counting

Definition: A Bash script is a set of commands executed by the Bash shell to automate tasks.
The lecture uses a file-counting task to demonstrate the importance of clear instructions when generating code.
Task: Create a Bash script that counts files in a directory.
A good prompt should specify the directory to check, what should count as a file, and how errors should be handled.

Illustrative example:

```
#!/bin/bash

directory="${1:-.}"

if [[ ! -d "$directory" ]]; then
    echo "Error: Directory not found."
    exit 1
fi

find "$directory" -maxdepth 1 -type f | wc -l
```

Explanation:

- The script accepts a directory as an argument.
- If no directory is supplied, it uses the current directory.
- It checks whether the directory exists.
- It counts regular files directly inside that directory, excluding subdirectories.

Note: This is an illustrative version of the lecture's file-counting task; the exact original script was not preserved in the extracted summary.

3. Hidden Assumptions and Safety Constraints

Definition: Hidden assumptions are requirements that a developer expects but has not explicitly stated in the prompt.
Example: A file-counting script might be expected to count only files in the selected directory, not files in its subdirectories.
A prompt should clarify:

- Whether subdirectories should be included.
- What to do if a directory does not exist.
- How to handle missing permissions.
- Whether files should be deleted, moved, or only read.
-

Safety Rule: Never assume that an AI-generated file operation is safe. Review the script before executing it, especially if it changes or deletes files.

4. Requesting Code Explanations

Definition: Code explanation prompting means asking an AI to explain existing code at a specified level of detail.

What to include:

1. Supply the complete code or relevant section.
2. Specify the explanation depth.

3. Identify particular lines or concepts to focus on.
4. Request optimization suggestions when needed.

Example:
```
Explain this Bash file-counting script.
Describe what each command does.
Explain the error handling and how the file count is
calculated.
```
Key Point: Specify whether you need a beginner-friendly overview or a detailed, line-by-line explanation.

5. Code Translation

Definition: Code translation is the process of converting code from one programming language to another while preserving its intended functionality.

Important requirements for a translation prompt:

- Provide the source code.
- Specify the source and target languages.
- Mention required language versions and libraries.
- Describe the expected behavior.
- Request equivalent error handling where appropriate.

Example from the lecture: Translating a Bash script that counts files into Python.

The goal is not merely to replace Bash commands with Python syntax. The translated program should preserve the original task's behavior.

6. Bash-to-Python Translation Example

Task: Convert the Bash file-counting script into Python.

Illustrative Python version:
```
from pathlib import Path
import sys

def count_files(directory="."):
    path = Path(directory)

    if not path.is_dir():
        raise NotADirectoryError(
            f"Directory not found: {directory}"
        )

    return sum(1 for item in path.iterdir() if
item.is_file())

if __name__ == "__main__":
```

```
    directory = sys.argv[1] if len(sys.argv) > 1 else "."

    try:
        print(count_files(directory))
    except (NotADirectoryError, PermissionError) as error:
        print(f"Error: {error}", file=sys.stderr)
        sys.exit(1)
```

Explanation:

- Path() represents the directory.
- is_dir() checks whether the path is a directory.
- iterdir() retrieves its immediate entries.
- is_file() selects regular files.
- sum() counts the selected files.
- The exception handler reports errors instead of allowing them to pass silently.

Key Point: Code translation should preserve the original intent, expected output, and important error-handling behavior.

The Python code above is an illustrative translation of the lecture's file-counting task, not a verified copy of the video's original code.

7. Applications of Code Translation

Code translation with AI can help developers:

- Convert scripts between programming languages.
- Port existing functionality to a new environment.
- Understand unfamiliar code.
- Reduce repetitive conversion work.
- Compare equivalent implementations.

Important: Always review translated code for correctness, compatibility, security, and performance.

AI is a programming assistant, not a replacement for developer verification.

8. Debugging with AI

Definition: Debugging is the process of finding, analyzing, and fixing problems in a program.

AI-assisted debugging uses prompts to provide the model with enough information to diagnose and suggest corrections.

What to provide in a debugging prompt:

1. Complete relevant code.
2. The exact error message.
3. The intended functionality.
4. Expected and actual behavior.
5. A request to identify and explain the fixes.

Example:

```
Review my file-counting script.

It should count regular files in a specified directory.
Here is the code and the error message.

Identify the cause of the error, explain the fix,
and provide the corrected code.
Do not remove the existing functionality.
```

Key Point: More relevant context helps the AI investigate the actual problem rather than guessing.

9. Debugging Complex Problems

Real-world debugging may involve multiple problems at the same time. A structured debugging prompt should ask the AI to:

- Identify the problems.
- Explain the likely causes.
- Recommend corrections.
- Check whether the corrections preserve the intended functionality.
- Suggest tests to verify the result.

Important: Do not apply every suggested fix blindly. Test the corrected program against the requirements and relevant edge cases.

10. Prompt Engineering Lessons for Coding

The lecture emphasizes these key practices:

Practice	Meaning
Context	Supply relevant code and task background.
Precise Instructions	State the expected behavior explicitly.
Constraints	Define language, dependencies, and limitations.
Error Handling	Explain how failures should be handled.
Safety	Avoid risky file operations without safeguards.
Human Validation	Review and test AI-generated code.

Quick Revision Sheet

Topic	Definition

Code Prompt	An instruction for generating or working with code.
Effective Code Prompt	A prompt with clear requirements and constraints.
Hidden Assumption	An unstated requirement or expectation.
Error Handling	Managing errors and exceptional conditions.
Code Explanation	Asking AI to describe how existing code works.
Code Translation	Converting code while preserving its functionality.
Debugging	Identifying and fixing program errors.
Human Validation	Verifying AI-generated code before relying on it.

Final Takeaway: Effective coding prompts clearly define the task, language, expected behavior, constraints, and error handling. Whether AI generates, explains, translates, or debugs code, the developer must validate the final result.